

Method Reference vs Static Method Call in Java 8

A **method reference** and a **static method call** may both involve static methods, but they serve **completely different purposes**.

- A **static method call** immediately executes a method.
 - A **method reference** does **not** execute the method immediately. Instead, it creates a reference that can be passed to another method (such as a Stream operation) and executed later.
-

Definition of a Static Method Call

A static method call directly invokes a static method using the class name.

Syntax

```
ClassName.methodName(arguments);
```

Example

```
public class MathUtil {  
  
    public static int square(int n) {  
        return n * n;  
    }  
  
    public static void main(String[] args) {  
  
        int result = MathUtil.square(5);  
  
        System.out.println(result);  
    }  
}
```

Output

25

Execution Flow

```
main()  
|  
▼  
MathUtil.square(5)  
|  
▼
```

Returns 25

The method executes immediately.

Definition of a Method Reference

A method reference is a shorthand notation for a lambda expression that refers to an existing method.

Instead of writing

```
x -> MathUtil.square(x)
```

you write

```
MathUtil::square
```

Notice:

- No method execution happens here.
 - Only a reference is created.
-

Syntax

```
ClassName::staticMethod
```

Example

```
import java.util.Arrays;
import java.util.List;

public class Demo {

    public static int square(int x) {
        return x * x;
    }

    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(1,2,3,4,5);

        numbers.stream()
```

```
        .map(Demo::square)
        .foreach(System.out::println);
    }
}
```

Output

```
1
4
9
16
25
```

How It Works Internally

This code

```
.map(Demo::square)
```

is converted by the compiler approximately into

```
.map(x -> Demo.square(x))
```

Every stream element becomes the parameter.

```
1
|
▼
square(1)
```

```
2
|
▼
square(2)
```

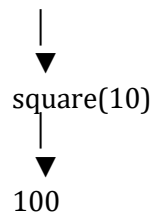
```
3
|
▼
square(3)
```

Static Method Call Example

```
int result = Demo.square(10);
```

Execution

```
main()
```



The method executes immediately.

Method Reference Example

```
Function<Integer,Integer> f = Demo::square;
```

Nothing executes yet.

Later,

```
f.apply(10);
```

Now execution occurs.

Side-by-Side Comparison

Feature	Static Method Call	Method Reference
Executes immediately	Yes	No
Stores a reference	No	Yes
Used with functional interfaces	No	Yes
Can be passed as an argument	No	Yes
Introduced in Java	Java 1.0	Java 8
Uses ::	No	Yes
Requires lambda target type	No	Yes

Example 1 — Static Method Call

```
public class Demo {  
  
    public static void hello() {  
        System.out.println("Hello");  
    }  
  
    public static void main(String[] args) {  
  
        Demo.hello();  
  
        System.out.println("Done");  
    }  
}
```

Output

Hello
Done

Execution

hello()
runs immediately

Example 2 — Method Reference

```
public class Demo {  
  
    public static void hello() {  
        System.out.println("Hello");  
    }  
  
    public static void main(String[] args) {  
  
        Runnable r = Demo::hello;  
  
        System.out.println("Before");  
  
        r.run();  
  
        System.out.println("After");  
    }  
}
```

Output

Before
Hello
After

Notice:

The method is **not executed** when assigning

```
Runnable r = Demo::hello;
```

It executes only when

```
r.run();
```

is called.

Lambda Equivalent

Method Reference

```
Runnable r = Demo::hello;
```

Equivalent Lambda

```
Runnable r = () -> Demo.hello();
```

Stream Example

Static Method Call (Not Valid)

```
numbers.stream()  
    .map(Demo.square());
```

Compilation Error

Why?

Because `map()` expects a function, not the result of calling a function.

Correct Method Reference

```
numbers.stream()  
    .map(Demo::square);
```

Lambda Equivalent

```
numbers.stream()  
    .map(x -> Demo.square(x));
```

Functional Interface Requirement

Method references always require a functional interface.

Example

```
Function<Integer,Integer> function = Demo::square;
```

Internally

```
Function  
  |  
  ▼  
apply(x)  
  |  
  ▼  
Demo.square(x)
```

Another Example

```
List<String> names =  
Arrays.asList("Alice","Bob","Charlie");  
  
names.forEach(System.out::println);
```

Equivalent Lambda

```
names.forEach(name -> System.out.println(name));
```

Equivalent Static Call? **No.**

You cannot write

```
System.out.println();
```

because `forEach()` needs a function to invoke for each element, not a single immediate call.

Real-World Analogy

Imagine a TV remote:

Static Method Call

Press the **Power** button.

Power ON

The action happens immediately.

Method Reference

Give someone the remote.

Remote handed over

Nothing happens until they press the button later.

When to Use a Static Method Call

Use a static method call when you want to:

- Execute a method immediately.
- Compute a value.
- Perform an action right away.
- Invoke utility methods like `Math.max()`, `Integer.parseInt()`, or `Collections.sort()` (before Java 8 enhancements).

Example

```
double value = Math.sqrt(81);
```

When to Use a Method Reference

Use a method reference when:

- Passing behavior to another method.
- Working with the Stream API.
- Using functional interfaces.
- Scheduling work for later execution.
- Replacing simple lambda expressions with more readable code.

Examples

```
list.forEach(System.out::println);
```

```
stream.map(Integer::parseInt);
```

```
stream.map(String::toUpperCase);
```

```
stream.sorted(String::compareToIgnoreCase);
```

Advantages of Method References

- Shorter and cleaner syntax.
 - Improves readability.
 - Reuses existing methods.
 - Reduces boilerplate code.
 - Integrates naturally with Java 8 functional programming and the Stream API.
-

Summary

Aspect	Static Method Call	Method Reference
Purpose	Execute a method immediately	Pass a method as behavior
Syntax	Class.method(args)	Class::method
Execution Time	Immediate	Deferred until invoked
Returns	Method result	Functional interface instance
Lambda Equivalent	Not applicable	Yes
Stream API Usage	No	Yes
Functional Interface Required	No	Yes
Common Use Cases	Utility methods, calculations	Streams, callbacks, event handlers, executors

Key Takeaways

- A **static method call** executes the method immediately and returns its result.

- A **method reference** creates a reference to an existing method without invoking it immediately.
- Method references are used with **functional interfaces** and are primarily intended for passing behavior, especially in the Java 8 Stream API and other functional programming features.
- A method reference like `Demo::square` is typically equivalent to the lambda expression `x -> Demo.square(x)`, but it is more concise and often more readable.

Method Reference vs Instance Method Call in Java 8

Although both **method references** and **instance method calls** involve object methods, they are fundamentally different.

- An **instance method call** executes a method immediately on a specific object.
- A **method reference** does **not** execute the method immediately. Instead, it creates a reference to an instance method that can be invoked later through a functional interface.

Definition of an Instance Method Call

An **instance method call** invokes a non-static method on an object.

Since the method belongs to an object (instance), you must create or obtain that object before calling the method.

Syntax

`object.methodName(arguments);`

Example

```
public class Calculator {

    public int square(int n) {
        return n * n;
    }

    public static void main(String[] args) {

        Calculator calculator = new Calculator();

        int result = calculator.square(5);

        System.out.println(result);
    }
}
```

Output

25

Execution Flow

```
main()
  |
  ▼
calculator.square(5)
  |
  ▼
Returns 25
```

The method executes immediately.

Definition of a Method Reference

A **method reference** is a compact way of referring to an existing method without calling it immediately.

Instead of writing a lambda expression

```
x -> calculator.square(x)
```

you can write

```
calculator::square
```

No method is executed when the reference is created.

Types of Instance Method References

Java supports two kinds of instance method references.

1. Reference to an Instance Method of a Particular Object

Syntax

```
object::instanceMethod
```

Example

```
Calculator calculator = new Calculator();
```

```
Function<Integer, Integer> function = calculator::square;
```

Equivalent Lambda

```
Function<Integer, Integer> function =  
    x -> calculator.square(x);
```

2. Reference to an Instance Method of an Arbitrary Object of a Particular Type

Syntax

ClassName::instanceMethod

Example

String::toUpperCase

Equivalent Lambda

```
str -> str.toUpperCase()
```

Notice that no object is specified. Java automatically uses the stream element (or functional interface receiver) as the object.

Example 1 – Instance Method Call

```
public class Person {  
  
    public void sayHello() {  
        System.out.println("Hello");  
    }  
  
    public static void main(String[] args) {  
  
        Person person = new Person();  
  
        person.sayHello();  
  
        System.out.println("Done");  
    }  
}
```

Output

Hello
Done

The method executes immediately.

Example 2 – Method Reference

```
public class Person {  
  
    public void sayHello() {  
        System.out.println("Hello");  
    }  
  
    public static void main(String[] args) {  
  
        Person person = new Person();  
  
        Runnable runnable = person::sayHello;  
  
        System.out.println("Before");  
  
        runnable.run();  
  
        System.out.println("After");  
    }  
}
```

Output

Before
Hello
After

The assignment

```
Runnable runnable = person::sayHello;
```

does not execute sayHello(). The method runs only when run() is called.

Internal Working

Method Reference

```
Runnable runnable = person::sayHello;
```

Compiler approximately converts it into

```
Runnable runnable = () -> person.sayHello();
```

The object is stored inside the generated lambda.

Example with Function

```
class Calculator {  
  
    public int square(int x) {  
        return x * x;  
    }  
}  
  
public class Demo {  
  
    public static void main(String[] args) {  
  
        Calculator calculator = new Calculator();  
  
        Function<Integer, Integer> function =  
            calculator::square;  
  
        System.out.println(function.apply(6));  
    }  
}
```

Output

36

Equivalent Lambda

```
Function<Integer, Integer> function =  
    x -> calculator.square(x);
```

Example with Streams

```
import java.util.Arrays;  
import java.util.List;  
  
public class Demo {  
  
    public static void main(String[] args) {
```

```
List<String> names =  
    Arrays.asList("alice", "bob", "charlie");  
  
names.stream()  
    .map(String::toUpperCase)  
    .forEach(System.out::println);  
}  
}
```

Output

```
ALICE  
BOB  
CHARLIE
```

Equivalent Lambda

```
names.stream()  
    .map(name -> name.toUpperCase())  
    .forEach(System.out::println);
```

Instance Method Call in a Loop

Without Streams

```
for (String name : names) {  
    System.out.println(name.toUpperCase());  
}
```

Here, `toUpperCase()` is called immediately for each object.

Stream with Method Reference

```
names.stream()  
    .map(String::toUpperCase);
```

The Stream framework invokes `toUpperCase()` later for each element during stream execution.

Why This Is Not Valid

```
names.stream()  
    .map(name.toUpperCase());
```

Compilation Error

Reason:

`map()` expects a function (`Function<T,R>`), not the result of calling a method.

Correct versions are:

```
.map(String::toUpperCase)
```

or

```
.map(name -> name.toUpperCase())
```

Functional Interface Requirement

Method references always target a functional interface.

Example

```
Function<String, String> function =  
    String::trim;
```

Equivalent Lambda

```
Function<String, String> function =  
    str -> str.trim();
```

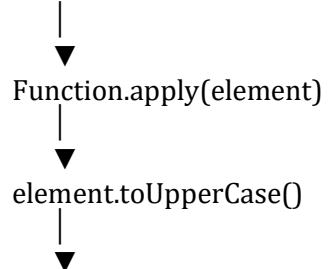
Internal Flow

For

```
names.stream()  
    .map(String::toUpperCase)
```

Execution

Stream Element



Returns uppercase String

Real-World Analogy

Imagine a television.

Instance Method Call

You pick up the remote and immediately press the **Volume Up** button.

Volume increases now.

Method Reference

You hand someone the remote.

Nothing happens.

Later they press **Volume Up**.

Volume increases then.

The remote represents the method reference, while pressing the button represents invoking the functional interface.

Advantages of Method References

- Cleaner and shorter than equivalent lambda expressions.
 - Improve readability.
 - Reuse existing methods without additional code.
 - Work naturally with Streams and functional interfaces.
 - Reduce boilerplate.
-

When to Use an Instance Method Call

Use an instance method call when you want to:

- Execute a method immediately.
- Update an object's state.
- Retrieve data directly from an object.
- Perform an action right away.

Example

```
employee.calculateSalary();
```

When to Use a Method Reference

Use a method reference when you want to:

- Pass behavior to another method.
- Work with the Stream API.
- Use callbacks or event handlers.
- Execute code later through a functional interface.
- Replace a simple lambda with more concise syntax.

Examples

```
list.forEach(System.out::println);
```

```
stream.map(String::trim);
```

```
stream.map(String::toLowerCase);
```

```
stream.filter(String::isEmpty);
```

```
stream.sorted(String::compareToIgnoreCase);
```

Method Reference vs Instance Method Call

Feature	Instance Method Call	Method Reference
Purpose	Execute an instance method immediately	Refer to an instance method for later execution
Syntax	object.method()	object::method or Class::instanceMethod
Executes immediately	Yes	No
Returns	Method result	Functional interface instance
Requires functional interface	No	Yes
Can be passed as an argument	No	Yes
Used with Streams	No	Yes
Lambda equivalent	Not applicable	Yes

Summary

- An **instance method call** invokes a method on an object immediately and returns its result.
- A **method reference** creates a reusable reference to an existing instance method without invoking it immediately.
- A method reference is always used in the context of a **functional interface** and is commonly used with Java 8 features such as the Stream API, callbacks, and asynchronous execution.
- `object::method` is typically equivalent to `args -> object.method(args)`, while `Class::instanceMethod` is typically equivalent to `obj -> obj.instanceMethod()`.

Method Reference vs Call to an Instance Method of an Arbitrary Object of a Particular Type

This is a subtle but important Java 8 concept.

Many developers confuse these two because they both involve **instance methods**, but they are **not the same thing**.

The key difference is:

- A **call to an instance method** executes the method immediately on an object.
- A **method reference to an instance method of an arbitrary object of a particular type** (`ClassName::instanceMethod`) does **not** execute the method immediately. It creates a function that Java can invoke later on whatever object is supplied.

Definition of a Call to an Instance Method

An instance method belongs to an object.

To call it, you need an object.

Syntax

```
object.instanceMethod(arguments);
```

Example

```
String name = "java";
```

```
String result = name.toUpperCase();
```

```
System.out.println(result);
```

Output

JAVA

Execution Flow

```
name
|
▼
toUpperCase()
|
▼
"JAVA"
```

The method is executed immediately.

Definition of a Method Reference

A method reference is a compact way of creating a function that refers to an existing method.

One of its forms is

ClassName::instanceMethod

Example

String::toUpperCase

Notice:

There is **no object** before ::.

The object will be supplied later by Java.

What Does "Arbitrary Object of a Particular Type" Mean?

Consider

String::toUpperCase

It means

"When someone gives me any String object, call toUpperCase() on that object."

The object is not known when the method reference is created.

It is supplied later.

Example

```
Function<String, String> function =  
    String::toUpperCase;
```

Equivalent Lambda

```
Function<String, String> function =  
    str -> str.toUpperCase();
```

Notice

String::toUpperCase

is **not**

someString.toUpperCase()

Immediate Call vs Method Reference

Immediate Instance Method Call

```
String name = "java";
```

```
String upper = name.toUpperCase();
```

Execution

```
name  
|  
▼  
toUpperCase()  
|  
▼  
JAVA
```

Runs immediately.

Method Reference

```
Function<String, String> function =  
    String::toUpperCase;
```

Nothing executes.

Later

```
function.apply("java");
```

Execution

```
"java"  
  |  
  ▼  
toUpperCase()  
  |  
  ▼  
JAVA
```

Internal Working

Suppose we write

```
Function<String, String> function =  
    String::trim;
```

The compiler approximately converts it into

```
Function<String, String> function =  
    str -> str.trim();
```

The object becomes the first parameter.

Stream Example

```
import java.util.Arrays;  
import java.util.List;  
  
public class Demo {  
  
    public static void main(String[] args) {  
  
        List<String> names =  
            Arrays.asList(  
                "alice",  
                "bob",
```

```
        "charlie");

    names.stream()
        .map(String::toUpperCase)
        .forEach(System.out::println);
    }
}
```

Output

```
ALICE
BOB
CHARLIE
```

What Happens Internally?

For every element

alice

Java executes

```
alice.toUpperCase()
```

For

bob

Java executes

```
bob.toUpperCase()
```

For

charlie

Java executes

```
charlie.toUpperCase()
```

Internally

```
.map(String::toUpperCase)
```

↓

```
.map(str -> str.toUpperCase())
```

Another Example

```
List<String> words =  
Arrays.asList(" Java ", " Python ");  
  
words.stream()  
    .map(String::trim)  
    .forEach(System.out::println);
```

Equivalent Lambda

```
words.stream()  
    .map(word -> word.trim())  
    .forEach(System.out::println);
```

Why Is This Called "Arbitrary Object"?

Suppose you have

`String::length`

Does Java know which String object?

No.

It could be

"Java"

or

"Python"

or

"AI"

or any other String.

Java waits until an object is supplied.

Example with Predicate

```
Predicate<String> predicate =  
    String::isEmpty;
```

Equivalent Lambda

```
Predicate<String> predicate =  
    str -> str.isEmpty();
```

Usage

```
System.out.println(predicate.test(""));  
System.out.println(predicate.test("Java"));
```

Output

```
true  
false
```

Example with Comparator

```
List<String> names =  
Arrays.asList(  
    "Bob",  
    "Alice",  
    "Charlie");  
  
names.sort(String::compareToIgnoreCase);  
  
System.out.println(names);
```

Output

```
[Alice, Bob, Charlie]
```

Equivalent Lambda

```
names.sort(  
    (a, b) ->  
        a.compareToIgnoreCase(b));
```

Here, the first argument (a) becomes the receiver object, and the second argument (b) is passed as the method argument.

Invalid Example

```
names.stream()  
    .map(name.toUpperCase());
```

Compilation Error

Why?

Because

`name.toUpperCase()`

returns a `String`.

But

`map()`

expects a function.

Correct

```
.map(String::toUpperCase)
```

or

```
.map(name -> name.toUpperCase())
```

Comparison

Instance Method Call

```
String text = "hello";
```

```
text.toUpperCase();
```

Known object

↓

Method executes immediately

Method Reference

```
String::toUpperCase
```

Unknown object

↓

Reference created

↓

Object supplied later

↓

Method executes

Internal Flow

Stream

↓

"java"

↓

Function.apply("java")

↓

"java".toUpperCase()

↓

JAVA

Real-World Analogy

Imagine a stamp.

Instance Method Call

You stamp **this document** immediately.

Document

↓

Stamp

↓

Completed

Method Reference

You tell someone:

"Whenever you receive a document, stamp it."

No document exists yet.

Later

Document arrives

↓

Stamp applied

That is exactly what

`String::toUpperCase`

means.

When to Use an Instance Method Call

Use an instance method call when you:

- Already have an object.
- Need the result immediately.
- Want to perform an operation directly.

Example

```
String value = text.trim();
```

When to Use `ClassName::instanceMethod`

Use it when:

- Passing behavior to APIs such as `Stream.map()`, `Stream.filter()`, or `List.sort()`.
- Working with functional interfaces like `Function`, `Predicate`, `Consumer`, `Comparator`, etc.
- Replacing simple lambda expressions with clearer code.

Examples

String::trim

String::toUpperCase

String::isEmpty

String::length

String::compareToIgnoreCase

Summary Table

Aspect	Instance Method Call	Method Reference (ClassName::instanceMethod)
Syntax	object.method()	ClassName::instanceMethod
Object Known at Creation	Yes	No
Executes Immediately	Yes	No
Returns	Method result	A functional interface implementation
Used in Streams	No	Yes
Lambda Equivalent	Not applicable	obj -> obj.instanceMethod() (or (obj, arg) -> obj.instanceMethod(arg) for methods with parameters)
Purpose	Perform an operation now	Pass behavior for later execution

Key Takeaways

- An **instance method call** executes immediately on a specific object that you already have.
- A **method reference to an instance method of an arbitrary object of a particular type** (ClassName::instanceMethod) does not invoke the method immediately. Instead, it creates a function where the **first parameter becomes the receiver object** when the function is invoked.
- For methods with no explicit parameters, String::trim is equivalent to s -> s.trim().
- For methods with one explicit parameter, String::compareToIgnoreCase is equivalent to (a, b) -> a.compareToIgnoreCase(b). This illustrates how the receiver object is supplied separately from the method's declared parameters.

Method Reference vs Constructor Reference in Java 8

A **method reference** and a **constructor reference** are both features introduced in **Java 8** to make code more concise by replacing simple lambda expressions. However, they serve different purposes.

- A **method reference** refers to an **existing method** (static or instance).
 - A **constructor reference** refers to a **constructor**, allowing objects to be created later through a functional interface.
-

Definition of a Method Reference

A **method reference** is a shorthand notation for a lambda expression that refers to an existing method without invoking it immediately.

It passes the method as behavior to a functional interface.

General Syntax

```
ClassName::staticMethod  
object::instanceMethod  
ClassName::instanceMethod
```

Example: Static Method Reference

```
import java.util.function.Function;  
  
public class Demo {  
    public static int square(int n) {  
        return n * n;  
    }  
  
    public static void main(String[] args) {  
        Function<Integer, Integer> function = Demo::square;  
        System.out.println(function.apply(5));  
    }  
}
```

Output

25

Equivalent Lambda

```
Function<Integer, Integer> function = x -> Demo.square(x);
```

Definition of a Constructor Reference

A **constructor reference** is a shorthand notation for referring to a constructor.

Instead of invoking the constructor immediately, Java creates a function that can construct objects later.

Syntax

```
ClassName::new
```

Example

```
class Student {  
    Student() {  
        System.out.println("Student Created");  
    }  
}  
  
public class Demo {  
    public static void main(String[] args) {  
        Supplier<Student> supplier = Student::new;  
        Student s = supplier.get();  
    }  
}
```

Output

Student Created

Equivalent Lambda

```
Supplier<Student> supplier = () -> new Student();
```

Main Difference

Method Reference

Refers to an **existing method**.

Demo::square

Equivalent

x -> Demo.square(x)

Constructor Reference

Refers to a **constructor**.

Student::new

Equivalent

() -> new Student()

Internal Working

Method Reference

Function<Integer,Integer> f = Demo::square;

Internally behaves approximately like

```
Function<Integer,Integer> f =  
    x -> Demo.square(x);
```

When executed

f.apply(10);

calls

Demo.square(10);

Constructor Reference

Supplier<Student> supplier =
 Student::new;

Internally behaves approximately like

```
Supplier<Student> supplier =  
    () -> new Student();
```

When executed

```
supplier.get();
```

calls

```
new Student();
```

Example with One Constructor Parameter

```
class Employee {  
  
    String name;  
  
    Employee(String name) {  
        this.name = name;  
    }  
}
```

Constructor Reference

```
Function<String, Employee> function =  
    Employee::new;  
  
Employee employee = function.apply("Alice");  
  
System.out.println(employee.name);
```

Output

Alice

Equivalent Lambda

```
Function<String, Employee> function =  
    name -> new Employee(name);
```

Example with Two Constructor Parameters

```
class Employee {  
  
    int id;  
    String name;  
  
    Employee(int id, String name) {  
  
        this.id = id;
```

```
    this.name = name;
  }
}
```

Constructor Reference

```
BiFunction<Integer, String, Employee> function =
    Employee::new;
```

```
Employee employee = function.apply(101, "Bob");
```

Equivalent Lambda

```
BiFunction<Integer,String,Employee> function =
    (id, name) -> new Employee(id, name);
```

Method Reference Example with Streams

```
import java.util.Arrays;

Arrays.asList("java", "python", "c++")
    .stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

Output

```
JAVA
PYTHON
C++
```

Equivalent Lambda

```
.map(s -> s.toUpperCase())
```

Constructor Reference with Streams

Suppose

```
class Student {

    String name;

    Student(String name) {
        this.name = name;
    }
}
```

```
List<String> names =  
    Arrays.asList("Alice", "Bob");
```

```
List<Student> students =  
    names.stream()  
        .map(Student::new)  
        .toList();
```

Equivalent Lambda

```
.map(name -> new Student(name))
```

For each String, the constructor is invoked to create a Student.

Types of Method References

Java supports four kinds of method references:

1. **Static method reference**

ClassName::staticMethod

2. **Instance method of a particular object**

object::instanceMethod

3. **Instance method of an arbitrary object of a particular type**

ClassName::instanceMethod

4. **Constructor reference**

ClassName::new

Although constructor references use the same :: syntax, they are considered a separate category because they refer to constructors rather than methods.

Real-World Analogy

Imagine a factory.

Method Reference

You already have a machine.

Machine

↓

Perform operation

Example

`String::trim`

The object already exists; you are asking it to perform an operation.

Constructor Reference

You ask the factory to build a new machine.

Factory

↓

Create new machine

Example

`Student::new`

No object exists yet; a new one is created when the functional interface is invoked.

When to Use a Method Reference

Use a method reference when you want to:

- Reuse an existing method.
- Simplify a lambda expression.
- Work with Streams and functional interfaces.
- Improve readability.

Examples

`String::trim`

`String::toUpperCase`

`Integer::parseInt`

System.out::println

When to Use a Constructor Reference

Use a constructor reference when you want to:

- Create objects through a functional interface.
- Build objects inside a Stream.
- Replace lambdas that only call a constructor.

Examples

Student::new

ArrayList::new

HashMap::new

Employee::new

Comparison Table

Feature	Method Reference	Constructor Reference
Refers to	Existing method	Constructor
Purpose	Invoke an existing method later	Create a new object later
Syntax	ClassName::method	ClassName::new
Lambda Equivalent	x -> method(x)	x -> new ClassName(x)
Returns	Method result	New object
Creates an object	No	Yes
Uses functional interfaces	Yes	Yes
Common Stream Usage	map(String::toUpperCase)	map(Student::new)

Summary

- A **method reference** points to an existing method (static or instance) and is used to invoke that method later through a functional interface.
- A **constructor reference** points to a constructor and is used to create new objects later through a functional interface.
- `Demo::square` is equivalent to `x -> Demo.square(x)`.
- `Student::new` is equivalent to `() -> new Student()` or `name -> new Student(name)`, depending on the constructor's parameters.
- Both features improve readability by replacing simple lambda expressions, but they solve different problems:
 - **Method reference** → perform an operation on existing objects or values.
 - **Constructor reference** → create new objects.

Lambda Expression vs Method Reference in Java 8

A **lambda expression** and a **method reference** are closely related features introduced in **Java 8** to support **functional programming**. A method reference is essentially a **more concise form of a lambda expression** when the lambda only calls an existing method.

Definition of a Lambda Expression

A **lambda expression** is an anonymous function that provides the implementation of a functional interface's single abstract method.

It has:

- No name
- No explicit return type (usually inferred)
- Can accept zero or more parameters
- Can return a value

Syntax

`(parameters) -> expression`

or

```
(parameters) -> {  
    // multiple statements  
}
```

Example

```
Function<Integer, Integer> square = x -> x * x;
```

```
System.out.println(square.apply(5));
```

Output

25

Definition of a Method Reference

A **method reference** is a shorthand syntax for a lambda expression that simply calls an existing method.

Instead of writing

```
x -> Integer.parseInt(x)
```

you can write

```
Integer::parseInt
```

A method reference does **not** execute the method immediately. It creates a reference that Java invokes later through a functional interface.

Relationship Between Them

A method reference is possible **only when the lambda does nothing except call an existing method**.

Example

Lambda

```
String::toUpperCase
```

is equivalent to

```
str -> str.toUpperCase()
```

Example 1 – Lambda Expression

```
List<String> names =  
    Arrays.asList("alice", "bob", "charlie");
```

```
names.stream()
    .map(name -> name.toUpperCase())
    .forEach(System.out::println);
```

Output

```
ALICE
BOB
CHARLIE
```

Example 2 – Method Reference

```
List<String> names =
    Arrays.asList("alice", "bob", "charlie");
```

```
names.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

Output

```
ALICE
BOB
CHARLIE
```

The behavior is the same; the method reference is simply shorter.

Internal Working

Lambda

```
name -> name.toUpperCase()
```

Method Reference

```
String::toUpperCase
```

The compiler treats them approximately the same in this case.

When Should You Use a Lambda Expression?

Use a lambda expression when you need **custom logic**, not just a direct method call.

Example 1: Multiple Operations

```
Function<String, String> function =  
    s -> s.trim().toUpperCase();
```

This cannot be replaced with a single method reference because it performs two operations.

Example 2: Conditional Logic

```
Predicate<Integer> even =  
    n -> {  
        if (n % 2 == 0) {  
            return true;  
        }  
        return false;  
    };
```

Equivalent simplified lambda

```
Predicate<Integer> even =  
    n -> n % 2 == 0;
```

A method reference cannot express this logic because there is no existing method that performs it.

Example 3: Combining Values

```
BiFunction<Integer, Integer, Integer> max =  
    (a, b) -> a > b ? a : b;
```

This requires new logic, so a lambda is appropriate.

When Should You Use a Method Reference?

Use a method reference when the lambda **only forwards its arguments to an existing method**.

Example 1

Lambda

```
x -> Integer.parseInt(x)
```

Method Reference

Integer::parseInt

Example 2

Lambda

name -> name.toUpperCase()

Method Reference

String::toUpperCase

Example 3

Lambda

x -> System.out.println(x)

Method Reference

System.out::println

Example 4

Constructor

Lambda

name -> new Student(name)

Constructor Reference

Student::new

Situations Where You Must Use a Lambda

1. Multiple Statements

```
Consumer<String> consumer = s -> {  
    System.out.println("Input: " + s);  
    System.out.println(s.toUpperCase());  
};
```

A method reference cannot represent multiple statements.

2. Calculations

```
Function<Integer, Integer> cube =  
    x -> x * x * x;
```

No existing method is being referenced.

3. Exception Handling

```
Consumer<String> consumer = s -> {  
    try {  
        Integer.parseInt(s);  
    } catch (Exception e) {  
        System.out.println("Invalid");  
    }  
};
```

A method reference cannot include try-catch blocks.

4. Complex Business Logic

```
Function<Employee, Double> salary =  
    e -> e.getSalary() * 1.10;
```

This computes a new value rather than simply invoking an existing method.

Situations Where a Method Reference Is Better

Printing

Lambda

```
list.forEach(x -> System.out.println(x));
```

Method Reference

```
list.forEach(System.out::println);
```

Parsing

Lambda

```
stream.map(s -> Integer.parseInt(s));
```

Method Reference

```
stream.map(Integer::parseInt);
```

Uppercase

Lambda

```
stream.map(s -> s.toUpperCase());
```

Method Reference

```
stream.map(String::toUpperCase);
```

Constructor

Lambda

```
stream.map(name -> new Student(name));
```

Constructor Reference

```
stream.map(Student::new);
```

Performance

There is generally **no meaningful performance difference** between a lambda expression and an equivalent method reference. Both are compiled to implementations that use the JVM's invokedynamic mechanism, and the JVM can optimize both effectively.

Choose between them based on **readability and clarity**, not performance.

Readability Comparison

Lambda

```
list.stream()  
  .map(s -> s.trim())  
  .map(s -> s.toUpperCase())
```

```
.forEach(x -> System.out.println(x));
```

Using Method References where appropriate

```
list.stream()  
    .map(String::trim)  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```

The second version is shorter and often easier to read.

Decision Guide

Situation	Use Lambda	Use Method Reference
Call an existing method only	✗	✓
Multiple statements	✓	✗
Conditional logic	✓	✗
Arithmetic or calculations	✓	✗
Create a new object	✗	✓ (ClassName::new)
Exception handling	✓	✗
Stream operations that simply call an existing method	✗	✓
Need custom business logic	✓	✗

Comparison Table

Feature	Lambda Expression	Method Reference
Introduced in	Java 8	Java 8
Purpose	Define behavior inline	Refer to an existing method or constructor
Syntax	(args) -> expression	Class::method, object::method, Class::new
Can contain multiple statements	✓ Yes	✗ No

Can include loops, conditions, try-catch	✓ Yes	✗ No
Can create new logic	✓ Yes	✗ No
Refers to an existing method	Not necessarily	✓ Yes
Supports constructors	Indirectly (<code>() -> new X()</code>)	✓ (<code>X::new</code>)
Used with functional interfaces	✓ Yes	✓ Yes
Readability for simple forwarding	Good	Usually better

Real-World Analogy

Imagine you're giving someone instructions.

Lambda Expression

You describe every step:

"Take the paper, fold it in half, sign it, and place it in the folder."

This is like writing custom logic.

Method Reference

You simply say:

"Use the **standard filing procedure**."

The procedure already exists; you're just referring to it.

Best Practices

- **Use a method reference** when a lambda simply calls an existing method or constructor. It usually makes the code shorter and easier to read.
- **Use a lambda expression** when you need to write custom logic, combine multiple operations, use conditionals, handle exceptions, or perform calculations.
- Don't force a method reference if it makes the code less clear. Readability is the primary goal.

Summary

- A **lambda expression** is an anonymous function that can contain any implementation needed for a functional interface.
- A **method reference** is a shorthand for a lambda that only invokes an existing method or constructor.
- Every method reference has an equivalent lambda expression, but **not every lambda expression can be converted into a method reference**.
- Prefer **method references** for simple delegation and **lambda expressions** for custom behavior and complex logic.

Can Method References Call Static and Instance Variables?

Short Answer:

No. A method reference cannot directly refer to a variable (static or instance). A method reference can refer **only to methods or constructors**.

However, a method reference **can indirectly access static or instance variables** if those variables are used inside the referenced method.

Definition of a Method Reference

A **method reference** is a shorthand syntax for a lambda expression that refers to an **existing method** or **constructor**.

It does **not** refer to:

- Variables
- Fields
- Constants
- Properties

It only refers to:

- Static methods
- Instance methods
- Constructors

What Can a Method Reference Reference?

✓ Static methods

Math::abs

✓ Instance methods of a particular object

System.out::println

✓ Instance methods of an arbitrary object of a particular type

String::toUpperCase

✓ Constructors

ArrayList::new

What It Cannot Reference

✗ Static variable

Demo::count

Compilation Error

✗ Instance variable

person::name

Compilation Error

Example 1 – Static Variable

```
class Demo {  
    static int count = 100;  
    public static void main(String[] args) {  
        System.out.println(Demo.count);  
    }  
}
```

Output

100

Trying a method reference

```
Function<?, ?> f = Demo::count;
```

Compilation Error

Reason:

count is a **field**, not a method.

Example 2 – Static Getter Method

Instead, create a method.

```
class Demo {  
  
    static int count = 100;  
  
    public static int getCount() {  
        return count;  
    }  
  
    public static void main(String[] args) {  
  
        Supplier<Integer> supplier =  
            Demo::getCount;  
  
        System.out.println(supplier.get());  
    }  
}
```

Output

100

Here the method reference is valid because it refers to the **method** `getCount()`, which internally accesses the static variable.

Example 3 – Instance Variable

```
class Employee {
```

```
    String name = "Alice";  
}
```

Accessing the field directly

```
Employee employee = new Employee();  
  
System.out.println(employee.name);
```

Output

Alice

Trying

```
employee::name
```

Compilation Error

Reason:

name is a field, not a method.

Example 4 – Instance Getter Method

```
class Employee {  
  
    String name = "Alice";  
  
    public String getName() {  
        return name;  
    }  
}
```

Now

```
Employee employee = new Employee();  
  
Supplier<String> supplier =  
    employee::getName;  
  
System.out.println(supplier.get());
```

Output

Alice

The method `getName()` accesses the instance variable internally.

Method References Indirectly Access Variables

```
class Counter {  
    static int total = 500;  
  
    public static int getTotal() {  
        return total;  
    }  
}
```

Method reference

```
Supplier<Integer> supplier =  
    Counter::getTotal;
```

Execution

```
supplier.get()
```

↓

```
Counter.getTotal()
```

↓

```
returns total
```

↓

```
500
```

The method reference calls the method, and the method reads the variable.

Another Example

```
class Student {  
    private String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
}
```

```
    public String getName() {  
        return name;  
    }  
}  
Student student = new Student("Bob");  
  
Supplier<String> supplier =  
    student::getName;  
  
System.out.println(supplier.get());
```

Output

Bob

Method Reference vs Lambda for Variables

Suppose

```
class Employee {  
  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
}
```

Lambda

```
Function<Employee, String> function =  
    employee -> employee.getName();
```

Method Reference

```
Function<Employee, String> function =  
    Employee::getName;
```

Both are equivalent because they reference the **getter method**, not the field.

Using a Static Variable in a Referenced Method

```
class Tax {  
  
    static double rate = 0.18;
```

```
    public static double calculate(double amount) {  
        return amount * rate;  
    }  
}
```

Method reference

```
Function<Double, Double> function =  
    Tax::calculate;  
  
System.out.println(function.apply(1000.0));
```

Output

180.0

The method reference invokes `calculate()`, which reads the static variable `rate`.

Using an Instance Variable in a Referenced Method

```
class Circle {  
  
    private double radius;  
  
    Circle(double radius) {  
        this.radius = radius;  
    }  
  
    public double area() {  
        return Math.PI * radius * radius;  
    }  
}  
Circle circle = new Circle(5);
```

```
Supplier<Double> supplier =  
    circle::area;  
  
System.out.println(supplier.get());
```

Output (approximately)

78.53981633974483

The `area()` method uses the instance variable `radius`.

Why Can't Method References Refer to Variables?

A method reference represents **executable behavior**.

A variable is **data**, not behavior.

For example:

Demo::getCount

means:

"When invoked, execute the getCount() method."

But

Demo::count

would mean:

"Refer directly to this field."

Java's method reference syntax does not support field references.

Comparison Table

Feature	Method Reference	Variable Access
Can reference a static method	✓ Yes	
Can reference an instance method	✓ Yes	
Can reference a constructor	✓ Yes	
Can directly reference a static field	✗ No	
Can directly reference an instance field	✗ No	
Can indirectly access fields through methods	✓ Yes	

Common Interview Questions

Can a method reference point to a variable?

No. It can only point to a method or constructor.

Can a method reference access a static variable?

Indirectly, yes. The referenced method can read or modify the static variable.

Can a method reference access an instance variable?

Indirectly, yes. The referenced method can read or modify the instance variable of the target object.

Can a method reference replace a getter?

No. It cannot replace the getter itself, but it **can reference the getter method**.

Example

Employee::getName

Best Practices

- Expose fields through methods (such as getters) if you want to use them with method references.
 - Avoid public mutable fields; encapsulation makes code safer and more maintainable.
 - Use method references when they clearly improve readability. If additional logic is needed, use a lambda expression instead.
-

Summary

- **Method references cannot directly reference static or instance variables.**
- **Method references can only reference methods or constructors.**
- A referenced method may internally **read or modify static or instance variables**, so variables are accessed **indirectly**, not directly.
- Examples:
 - ✓ Employee::getName
 - ✓ Demo::getCount
 - ✗ Employee::name
 - ✗ Demo::count

In other words, **method references are references to executable behavior (methods/constructors), not to data (fields/variables).**